

Kubernetes Volumes

attaching disk storage to containers

Kubernetes in Action · Chapter 6

minikube 4.5.2 (1 CP + 3 Worker) · Calico CNI · Podman

오늘 다룰 내용

#	주제	핵심
1	emptyDir	컨테이너 간 파일 공유
2	hostPath	노드 로컬 영속
3	hostPath 한계	노드 이동 시 데이터 유실
4	직접 지정의 문제	이식성 없음
5	PV/PVC	역할 분리
6	StorageClass	Dynamic Provisioning

6장은 "문제 → 해결 → 새 문제 → 더 나은 해결"의 연쇄입니다

왜 Volume이 필요한가?

컨테이너 파일시스템의 근본적 한계

컨테이너 재시작 = 데이터 소멸

[Pod 생성]

Container 시작 → 이미지의 파일만 보임 (깨끗한 상태)
앱이 /data/에 파일 기록

[Container crash → 재시작]

새 Container 시작 → 이미지의 파일만 보임
이전 /data/ 파일? 사라짐! ❌

- 컨테이너 파일시스템은 이미지 레이어에서 온다
- 재시작하면 이전 컨테이너가 쓴 파일은 모두 소멸
- **Volume**이 이 문제를 해결한다

Volume의 기본 특성

특성	설명
Pod에 종속	독립 K8S 오브젝트가 아님
mount 필수	정의만으로는 사용 불가
여러 컨테이너 공유	같은 Pod의 모든 컨테이너가 mount 가능
여러 Volume 동시 사용	한 Pod에 여러 타입 가능

```
spec:
  volumes:                # Pod 레벨에서 Volume 정의
  - name: html
    emptyDir: {}
  containers:
  - name: web-server
    volumeMounts:         # 컨테이너 레벨에서 mount
    - name: html
      mountPath: /usr/share/nginx/html
```

1. emptyDir

Sidecar 패턴으로 컨테이너 간 파일 공유

emptyDir — 특징

항목	내용
초기 상태	비어있는 디렉토리
Lifecycle	Pod과 동일 — Pod 삭제 시 소멸
컨테이너 재시작	유지됨 (Volume은 Pod 수준)
주 용도	Sidecar 패턴, scratch space
Memory medium	medium: Memory → tmpfs (빠르지만 메모리 limit에 포함)

```
+--- Pod: fortune -----+
|
| [html-generator]          [web-server]
| 10초마다 fortune 생성      Nginx가 HTML 서빙
| → /var/htdocs/index.html   ← /usr/share/nginx/html
|
|           +---- Volume: html (emptyDir) ----+
|
+-----+
```

Demo: emptyDir Sidecar

```
apiVersion: v1
kind: Pod
metadata:
  name: fortune
spec:
  containers:
    - name: html-generator
      image: luksa/fortune
      volumeMounts:
        - name: html
          mountPath: /var/htdocs
    - name: web-server
      image: nginx:alpine
      volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
          readOnly: true
  volumes:
    - name: html
      emptyDir: {}
```


Demo: emptyDir 결과

```
$ kubectl get pod fortune
NAME      READY   STATUS    RESTARTS   AGE
fortune   2/2     Running   0           17s

$ curl http://localhost:8080
You will be attacked by a beast who has the body of a wolf,
the tail of a lion, and the face of Donald Duck.

# 10초 후 - fortune이 갱신됨!
$ curl http://localhost:8080
Live in a world of your own, but always welcome visitors.
```

html-generator가 쓰면 web-server가 읽는다. 두 컨테이너가 emptyDir를 통해 데이터를 공유하는 Sidecar 패턴.

2. hostPath

노드 파일시스템 직접 마운트

hostPath — emptyDir과 비교

항목	emptyDir	hostPath
Pod 삭제 시	소멸 ✗	보존 ✓
컨테이너 재시작	유지	유지
노드 이동 시	-	유실 ✗
DB 적합성	부적합	부적합

적절한 사용 사례: **DaemonSet**의 시스템 레벨 작업만

- Fluentd: `/var/log` , `/var/lib/docker/containers`
- kube-proxy: iptables 규칙 관리

규칙: 앱 데이터 저장에는 절대 hostPath를 쓰지 않는다

Demo: hostPath — 같은 노드에서 영속

```
$ kubectl apply -f mongodb-pod-hostpath.yaml # nodeSelector: minikube-m02

# 데이터 기록
$ kubectl exec mongodb -- mongosh --quiet --eval '
  db.getSiblingDB("mystore").quotes.insertOne({text: "hostPath test"});
'

# Pod 삭제 후 같은 노드에 재생성
$ kubectl delete pod mongodb
$ kubectl apply -f mongodb-pod-hostpath.yaml

# 데이터 확인 — 여전히 존재!
$ kubectl exec mongodb -- mongosh --quiet --eval '
  db.getSiblingDB("mystore").quotes.find().forEach(printjson);
'
{ text: 'hostPath test', createdAt: ISODate('2026-05-09T07:48:26.846Z') }
```

Pod 삭제 후에도 노드의 /tmp/mongodb에 파일이 남아있다

3. hostPath의 한계

다른 노드에서 데이터 유실

Demo: 노드 변경 시 데이터 유실

```
# 의도적으로 다른 노드(m03)에 배포
$ kubectl apply -f mongodb-pod-hostpath-m03.yaml

$ kubectl get pod mongodb -o wide
NAME      READY   STATUS    RESTARTS   NODE
mongodb   1/1     Running   0          minikube-m03

# 이전 데이터 확인
$ kubectl exec mongodb -- mongosh --quiet --eval '
  db.getSiblingDB("mystore").quotes.find().forEach(printjson);
'
(empty) ← 데이터 없음! ❌
```

```
Node m02 /tmp/mongodb: [데이터 있음] ← 이전 Pod이 여기 저장
Node m03 /tmp/mongodb: [비어있음]   ← 새 Pod은 여기를 봄
```

결론: 노드 장애, 스케일 조정으로 Pod이 이동하면 데이터를 잃는다. DB에 hostPath는 금지!

4. 외부 스토리지 직접 지정

이식성 없는 해법의 문제

클라우드 디스크를 Pod에 직접 명시

```
spec:
  volumes:
  - name: mongodb-data
    gcePersistentDisk:
      pdName: mongodb
      fsType: ext4
```

← GCE에서만 동작!
← 디스크 이름을 알아야 함!

이 방식의 3가지 문제:

문제	설명
이식성 없음	GKE 전용. AWS로 옮기면 YAML 전부 수정
역할 혼재	개발자가 디스크 이름, NFS 주소를 알아야 함
재사용 불가	dev/staging/prod마다 디스크 이름이 다름

⚠ Kubernetes의 근본 철학(인프라 추상화)에 위배!

문제의 본질

```
+-----+
| Pod YAML (개발자가 작성해야 함) |
| gcePersistentDisk:               |
|   pdName: mongodb                |
|   fsType: ext4                   |
+-----+
```

← GCE에서만 동작!
← 인프라 관리자만 아는 정보!
← 개발자가 왜 이것 알아야 하지?

우리가 원하는 것:

```
+-----+
| Pod YAML (개발자가 작성) |
| persistentVolumeClaim:    |
|   claimName: mongodb-pvc  |
+-----+
```

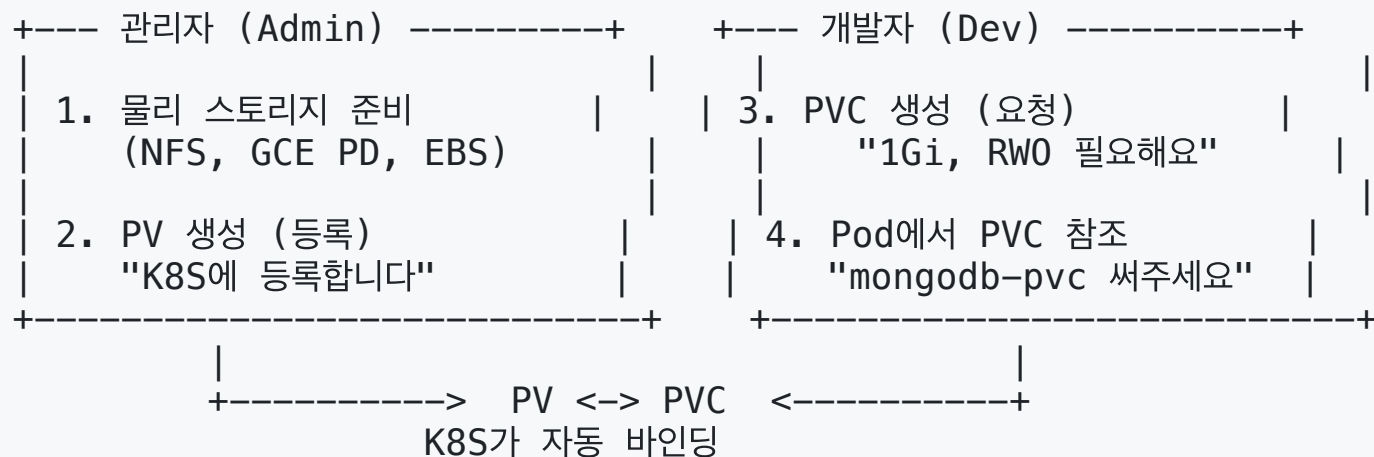
← "나는 스토리지가 필요합니다"
← 기술 세부사항? 몰라도 됨!

이것이 **PV/PVC** 패턴이 해결하는 문제입니다

5. PV/PVC

역할 분리 — 관리자와 개발자

PV/PVC 개념



핵심: 개발자는 ****"얼마나, 어떤 모드로"****만 요청. 실제 스토리지 기술은 몰라도 된다.

AccessMode와 Reclaim Policy

AccessMode — Node 수 기준 (Pod 수 아님!)

약어	의미	주의
RWO	단일 노드에서 R/W	같은 노드 여러 Pod은 OK
ROX	다수 노드에서 ReadOnly	
RWX	다수 노드에서 R/W	NFS, CephFS만

Reclaim Policy — PVC 삭제 후 PV 처리

Policy	동작	새 PVC 바인딩
Retain	데이터 보존, Released 상태	불가
Delete	PV + 스토리지 삭제	-

Demo: PV 생성 (관리자)

```
$ kubectl apply -f mongodb-pv.yaml
persistentvolume/mongodb-pv created
```

```
$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS
mongodb-pv	1Gi	RWO	Retain	Available

```
# mongodb-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mongodb-pv
spec:
  capacity:
    storage: 1Gi
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /tmp/mongodb-pv    # 인프라 세부사항은 PV에만!
```

Demo: PVC 생성 (개발자) → 자동 바인딩

```
$ kubectl apply -f mongodb-pvc.yaml
```

```
$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
mongodb-pvc	Bound	mongodb-pv	1Gi	RWO

← 자동 바인딩!

```
$ kubectl get pv
```

NAME	STATUS	CLAIM
mongodb-pv	Bound	default/mongodb-pvc

← Available → Bound

```
# mongodb-pvc.yaml - 인프라 정보 없음!
```

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: mongodb-pvc
```

```
spec:
```

```
  accessModes: [ReadWriteOnce]
```

```
  resources:
```

```
    requests:
```

```
      storage: 1Gi
```

```
  storageClassName: "" # 기존 PV에서 바인딩
```

K8S가 용량(1Gi)과 AccessMode(RWO) 매칭하여 자동 바인딩

Demo: Pod에서 PVC 사용 — 데이터 영속

```
$ kubectl apply -f mongodb-pod-pvc.yaml
$ kubectl exec mongodb-pvc -- mongosh --quiet --eval '
  db.getSiblingDB("mystore").quotes.insertOne({text: "PV/PVC: hidden"});
'

# Pod 삭제 후 재생성
$ kubectl delete pod mongodb-pvc
$ kubectl apply -f mongodb-pod-pvc.yaml

# 데이터 여전히 존재!
$ kubectl exec mongodb-pvc -- mongosh --quiet --eval '
  db.getSiblingDB("mystore").quotes.find().forEach(printjson);
'
{ text: 'PV/PVC: hidden', createdAt: ISODate('...') }
```

```
# Pod YAML - claimName만 참조!
volumes:
- name: mongodb-data
  persistentVolumeClaim:
    claimName: mongodb-pvc    # 인프라 세부사항 ZERO
```

PV/PVC 남은 한계

✓ 해결된 것:

- 개발자가 인프라 기술을 몰라도 됨
- Pod YAML의 이식성 확보
- 환경별 YAML 수정 불필요

✗ 아직 남은 문제:

- 관리자가 **PV**를 매번 수동으로 생성해야 함
- PVC 수백 개 → PV 수백 개를 미리 준비?
- 관리자의 반복 작업 부담이 큼

이 문제를 해결하는 것이 **Dynamic Provisioning + StorageClass**

6. Dynamic Provisioning

StorageClass로 완전 자동화

StorageClass란?

"어떤 provisioner가 어떤 파라미터로 스토리지를 생성할지" 정의

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-ssd
```

Dynamic Provisioning 흐름:

개발자: PVC 생성 (storageClassName: fast)

- K8S: "fast" StorageClass 확인
- Provisioner: 스토리지 자동 생성
- PV 자동 생성
- PVC에 자동 바인딩
- Pod에서 사용 가능!

관리자는 StorageClass 한 번만 정의. 이후 PV 생성은 완전 자동.

storageClassName 동작 정리

PVC 설정	동작
미지정 (필드 없음)	default StorageClass로 동적 프로비저닝
"fast" 등 지정	해당 StorageClass로 동적 프로비저닝
"" (빈 문자열)	동적 프로비저닝 비활성화 → 기존 PV에서만 바인딩

```
$ kubectl get storageclass
NAME                                PROVISIONER                RECLAIMPOLICY
standard (default)                 k8s.io/minikube-hostpath   Delete
```

(default) 표시 = storageClassName 미지정 시 자동 적용

Demo: PVC만 생성 → PV 자동 생성

```
$ kubectl apply -f mongodb-pvc-dynamic.yaml

# PV를 수동으로 만들지 않았는데 자동 생성!
$ kubectl get pvc
NAME                                STATUS    VOLUME                                STORAGECLASS
mongodb-pvc-dynamic                Bound    pvc-d4b8c3f3-...-2cbd                standard

$ kubectl get pv
NAME                                RECLAIM POLICY    STATUS    STORAGECLASS
pvc-d4b8c3f3-...-2cbd              Delete            Bound    standard
```

```
# PVC - storageClassName 미지정 → default(standard) 사용
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongodb-pvc-dynamic
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
```

Demo: PVC 삭제 → PV 자동 삭제

```
# 데이터 확인
$ kubectl exec mongodb-dynamic -- mongosh --quiet --eval '
  db.getSiblingDB("mystore").quotes.find().forEach(printjson);
'
{ text: 'Dynamic Provisioning: no manual PV needed!' }

# 정리 - PVC 삭제 시 PV도 자동 삭제!
$ kubectl delete pod mongodb-dynamic
$ kubectl delete pvc mongodb-pvc-dynamic

$ kubectl get pv
No resources found      ← Reclaim Policy: Delete
```

PVC 삭제 → PV 자동 삭제. 라이프사이클까지 자동 관리.

전체 요약

4단계 진화 흐름

6장 Volume — 진화 요약

단계	방식	문제 → 다음 단계로
1	emptyDir	Pod 삭제 시 데이터 소멸
2	hostPath	다른 노드 이동 시 데이터 유실
3	직접 지정 (gcePD/EBS)	YAML에 인프라 종속, 이식성 없음
4	PV/PVC	역할 분리 성공, PV 수동 생성 필요
5	StorageClass	✅ 완전 자동화 — 최종 해법

역할 분리 최종 모습:

개발자: PVC ("1Gi, RW0 필요") + Pod ("mongodb-pvc를 /data/db에 mount")
관리자: StorageClass ("standard = hostpath provisioner") – 한 번만!
K8S: PV 자동 생성 → 바인딩 → 사용 가능

한 문장 핵심

Pod에 스토리지 기술을 직접 지정하지 말고, PVC로 요청하고 StorageClass로 자동 프로비저닝하라.

개발자는 "얼마나 필요한지"만 말하면 된다.

Q&A

예상 질문:

질문	답변
RWO인데 여러 Pod이 쓸 수 있다?	RWO는 Node 수 제한. 같은 노드 여러 Pod은 OK
Released PV 재사용은?	PV 삭제 후 재생성 필요 (보안상 직접 재바인딩 불가)
StorageClass 보통 몇 개?	2~3개 (fast/standard/slow)
emptyDir Memory medium 주의점?	컨테이너 메모리 limit에 포함 → OOM 가능